

A Checklist for Evaluation of Reference Architectures for Embedded Systems

José Filipe Marreiros Santos*, Milena Guessi*, Matthias Galster[†], Daniel Feitosa* and Elisa Yumi Nakagawa*

*Department of Computer Systems
University of São Paulo - USP
PO Box 668, 13560-970, São Carlos, SP, Brazil
Email: joesantos@grad.icmc.usp.br,
{milena, fdaniel, elisa}@icmc.usp.br

[†] University of Groningen
Department of Mathematics and Computing Science
PO Box 407, 9700 AK Groningen, The Netherlands
Email: mgalster@ieee.org

Abstract—Embedded systems are programmable computers designed to perform specialized tasks and usually present stringent requirements, since, many times, their failure may result in human harm or environmental damage. In another perspective, reference architectures are reusable software engineering artifacts introduced to facilitate the design of software architectures of a given domain. In this sense, reference architectures increase the software quality by standardizing and promoting good practices for software development. To make sure that reference architectures capture all required information, it is important to have means of evaluating it. In this context, this work introduces a checklist approach for evaluation of reference architectures of embedded systems which helps to uncover defects in an early stage of the reference architecture life cycle. We elaborate on a web based tool that could support the checklist adoption in a collaborative, distributed environment. We also successfully applied the checklist in an academic reference architecture project. As main result, we hope to contribute to the evaluation of reference architectures of embedded systems. We also intend that this work could open interesting, new research perspectives in this direction. To validate this approach, we considered the opinion of experts in software architecture.

I. INTRODUCTION

Embedded systems can be understood as devices that include a programmable computer but are not itself intended to be a general-purpose computer [?]. Usually, the development of these systems requires more effort due to the more stringent requirements that may exist [?] (adaptability, maintainability, safety, security, and dependability, as well as business requirements, such as smaller time-to-market and lower cost, systems whose failure might harm human life, lead to substantial economic loss, or even cause extensive environmental damage [?]). Taking the automotive industry, for instance, it is possible to observe that modern cars are increasingly equipped with features

ranging from motor control to entertainment features, which heavily depend on embedded systems [?]. In addition, there are also a number of safety-critical functionalities that must be accomplished, such as for advanced driver assistance, motor control, and vehicle stability.

We have observed that currently the major role of software architectures is to determine the quality of software systems such as maintainability, evolvability, dependability, and interoperability, as also previously mentioned in [?]. These software architectures comprise the structure of components of a system, their interrelationships, and principles and guidelines governing its design and evolution over time [?]. In order to enable their effective use in the creation, maintenance, and evolution of software systems, it is essential to properly describe the software architectures in a way that all their stakeholders (e.g., customers, developers, and software architects) can understand and actually use them. In this context, we can observe the use of Architectural Description Languages (ADLs), such as Wright [?] and Rapide [?], which enable to describe architectures based on formally defined syntax and semantics. Besides that, UML (Unified Modeling Language) [?] enables a semi-formal description of architectures and has been increasingly adopted for the architectural description since its version 2.0 [?].

In the context of software architecture, we observe the proposition of reference architecture, which aim at facilitating and systematizing the development of set of software systems of a given domain. In particular, reference architectures refer to an architecture that encompasses the knowledge about how to design concrete architectures of systems of a given application domain; therefore, it must address the business rules, architectural styles (sometimes also defined as architectural patterns that address quality

attributes in the reference architecture), best practices of software development (for instance, architectural decisions, domain constraints, legislation, and standards), and the software elements that support development of systems for that domain [?].

Given the diversity of requirements and the safety criticality, the design of embedded systems demands a quality driven approach, which can be provided by the use of reference architectures. In fact, the adoption of reference architectures for the embedded systems domain brings many benefits such as [?]: improving integrability, supporting design decisions reuse, standardizing basic software, enabling easier portability among platforms, improving description understandability by abstracting important architecture functionalities and features, formalizing the process of designing the architectures from this domain, and also for clarifying the architecture requirements.

Considering that reference architectures are the basis for the construction of a set of systems, it is important to make sure that all the concerns were addressed in its description so it can also serve as an effective communication vehicle throughout systems development.

The main objective of this paper is to present a checklist for evaluation of reference architectures of embedded systems. The checklist approach was evaluated by experts in software architecture and was also successfully applied in the context of an academic research work. We also identify future and important research lines based on our results.

This paper is organized as follows. In Section ?? we present the necessary background for our research. In Section ?? we present the checklist proposal and a tool to support its application. In Section ?? we present the results of a qualitative evaluation of the checklist by experts on software architecture and reference architecture. An evaluation of a reference architecture using the checklist is also presented in this section. Finally, in Section ?? we summarize our contributions and discuss perspectives for future work.

II. BACKGROUND

Embedded systems have very rigorous requirements, mainly safety and business related. Given that standardization can help improve their quality, embedded systems is application domain that can benefit from the use of reference architectures [?]. One example of embedded systems reference architecture is AUTOSAR (AUTomotive Open System ARchitecture),¹ a reference architecture for systems embedded in cars. AUTOSAR was created by the main car and parts manufacturers (BMW, Bosch, Toyota, Continental, Daimler, Ford, GM, Peugeot, Volkswagen) to standardize interfaces and stimulate innovation. Other example is NASA

ESDS² reference architecture, it was created to suppress the need for a standardized approach to deal with geophysical data. The NASA ESDS reference architecture describes the resources and standards of the existing systems and provides guidelines for the planing, construction and operations of new of new software systems in this domain.

The ISO 42010 [?] provides rules, definitions, and recommendations on organization and representation of software architecture. This promotes the communication between stakeholders by avoiding the issues caused by an ad hoc approach and yet, not binding the architects to a very strict set of rules. (For example, the standard contains definitions and rules about viewpoints, views, models, rationale and general information).

In the context of reference architectures, evaluation methods have been investigated in order to improve the final software artifacts quality, among the proposed methods stand the SAAM (Software Architecture Analysis Method) [?] and ATAM (Architecture Trade-off Analysis Method)³. There is, however, a discussion on what questions should be treated by the chosen method and which method is more suitable for each situation [?]. Besides that, a detailed and complex evaluation demands more resources. In this paper, we present a evaluation approach based on the checklist technique that is a flexible and simple method that can be used alone or as a pre-evaluation along with other evaluation methods, in order to improve the quality of embedded system reference architectures and promote discussions that may increase the understanding of the documentation by all the stakeholders. Given that the checklist approach can be a simple, cheap, and flexible way to perform a high level evaluation, it can save a great deal of resources and time by unmasking defects and clearing doubts at an early stage of the reference architecture life-cycle.

III. CHECKLIST DEFINITION

The checklist construction was guided by literature available on embedded systems, reference architectures and software architecture. Through the understanding of these topics, we identified important characteristics and required elements in a embedded systems reference architectures and incorporated them in the checklist. At first, we adapted the checklist for software architectures initially provided by Celemnts et al. [?] for the reference architecture. This initial checklist already had questions related to conformance with the ISO 42010. Next, we extended the checklist with questions related to quality attributes by Rozanski et al. [?]. The checklist construction also incorporated the work of [?]

¹Available at <http://www.autosar.org/> (Accessed on 03/10/2013)

²Available at <http://earthdata.nasa.gov/sites/default/files/field/document/ESDS\%20Reference\%20Architecture\%20v1.1.pdf> (Accessed on 03/10/2013)

³Available at <http://www.sei.cmu.edu/architecture/tools/evaluate/atam.cfm> Accessed on (03/10/2013)

a list of characteristics for a successful checklist were provided. Finally, the checklist was refined to cover the topics treated by Nakagawa et al. [?] in their RAModel, which contained a set of information that reference architectures should provide.

Seven stakeholders roles are considered: architects, domain experts, developers, analysts, software manager, integrators, testers and quality assurance stakeholders. The architects are the software architecture specialists, they are responsible for designing the reference architecture and deriving it. The domain experts are, in this case, the ones with knowledge on embedded systems, or, more specifically on the subfield that the reference architecture is being designed for. The developers are the professionals responsible for implementing the application. The analysts are in charge of the documentation i.e. requirements documents, UML diagrams. The software manager makes the strategic decision of the project, like resources and time managements. The integrators are responsible for the system consistency and adaptation. Testers are in charge of finding defects on the developed application. Finally, quality assurance stakeholders are any stakeholders who verify the project conformance with the quality requirements. These considerations were made by Clements et al. [?] and we kept this organization. Questions targeting stakeholders that are not known or do not exist, depending on the abstraction level of the reference architecture, can simply be ignored.

In addition, nine concerns about embedded systems are considered: availability, interoperability, maintainability, performance, security, reliability, scalability, protection, usability. These concerns were determined by a systematic review by Guessi et al. [?].

Our checklist proposal is structured into four stages, namely (i) General Information, (ii) Raising Discussions, (iii) General Analysis Conclusion and (iv) Embedded Systems Specifics. These stages are divided into one or more sets. The first stage, for example is divided into two sets, in the first set there are questions about general information, i.e. overview data, viewpoints, views, models; while the second set contains questions about the checklist construction i.e. if quality attributes were considered or if the stakeholders are defined. The stakeholders to respond to this stage are solely the architects. This stage has 25 questions, 9 in the first set and 16 in the second. In the sequence, the second stage, has only one set with questions to all stakeholders roles with the objective of raising discussions, aiming to improve the understanding of design, rationale and clear any doubts. There are 52 questions in the second stage, 7 for all stakeholders altogether, 12 for the architects, 6 to be answered by the domain experts, 6 for the project manager, 4 for the developers, 3 for integrators, 3 for the testers, 9 for the quality assurance managers and 2 for the analysts. The third stage has only one set with

3 questions about general analysis conclusion, i.e. if the reference architecture fits its purpose. This stage is targeted to all stakeholders while the fourth and final stage has two sets to be answered by the embedded systems experts. There are 11 questions in the first set about embedded hardware and 3 about embedded software. The evaluation team, however, can work with only the questions that seem to be suitable for the ongoing project. Table ?? presents the subjects by stage and table ?? presents a sample of the checklist.

We recommend the checklist application during the elaboration of the reference architecture. At this point, it should catch defects when modification on the reference architecture are cheaper than in a later stage of the reference architecture life cycle.

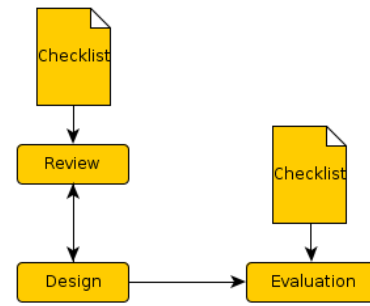


Fig. 1. The checklist application points in a general life cycle. The checklist can be applied during design reviews or during the final evaluation

In addition, the checklist can be used as a pre-evaluation method. If another evaluation is required, then checklist can be used to minimize the effort of more detailed evaluation methods.

A. Supporting Tool

Currently, FERA-ES (Framework for evaluation of reference architectures of embedded systems), an open source tool for the checklist application support is under development to assist the checklist appliance. The growing number of worldwide distributed teams and the lack of a centralized tool for the checklist application motivated the decision to implement the it as a web based system. FERA-ES is being designed to be flexible in the sense that any checklist for any kind of evaluation can be managed through it. The objective is to provide an online, collaborative environment for evaluation control and results management. The evaluation team will be able to create new projects and insert the checklist as we proposed or customized into these projects. The checklists will be separated by stakeholders or role, so when the respondent logs in to answer, he will be granted access only to the questions that are specific for their role in the reference architecture.

TABLE I
SUBJECTS AND NUMBER OF QUESTIONS IN EACH STAGE

Stage	Set	Questions	Subjects covered
1	1	9	overview information, viewpoints, views, models, stakeholders and concerns
	2	16	legal regulation, ISO 42010 conformance, design and development issues, domain specific issues, conformance with other artifacts, quality attributes
2	1	52	specific questions to stakeholders
3	1	3	conclusion of the general analysis
4	1	11	embedded hardware
	2	3	embedded software

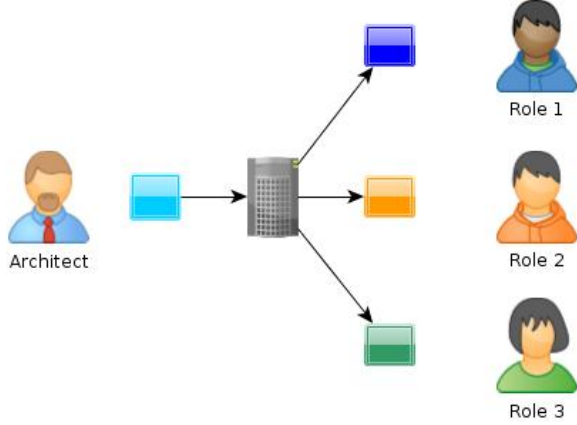


Fig. 2. Checklist access control according to stakeholder and roles

The checklist control will allow deadlines creation and edition, questions edition and respondent specification, i.e. which respondent will be granted access to answer the checklist. Considering the respondent perspective, they will be able to change and review their answers until the deadline.

FERA-ES will support results management through data visualization techniques, making the results easier to understand and saving time and effort in the reference architecture evaluation since it helps in the understanding the current state of the evaluation and pointing out which parts require more attention. The last feature included is the collaboration support through comments on each question. This feature is essential for the applicability of FERA-ES since the checklist application requires stakeholders to discuss and collaborate among each other.

Regarding technical decisions about the implementation, we made the choice to use only free technologies. Such technologies are robust and compatible with the free software philosophy. We are also using the prototyping software process in development, since simple prototypes can be built in a short period of time with minimum effort, we

can include or change functionalities without causing delays in the development. Once the prototypes are validated, we will create an online open source code repository and start to code the application. This validation will be made by reference architecture experts in our research group. The validation is important because it will give a qualitative idea of the applicability of FERA-ES.

IV. EVALUATION REPORT

In order to validate the checklist proposal, software architecture experts were consulted. They provided insights about the checklist in construction and re-evaluation cycles. After finished, a qualitative was conducted by a team of experts of our research group. These experts have been involved with software architecture, reference architecture, and embedded system in their research. Each one of them answered a questionnaire, in which, each respondent was asked about the benefits they suspect the use of the checklist will offer, and also, to provide insights on the checklist characteristics.

The answers collected showed us that the checklist questions are clear, their enunciates are written without ambiguities and the support material along the checklist is enough for the evaluation team to understand all the technical terms. Results also pointed that the checklist have a broad topic coverage, treating all important topics that play a role in the reference architecture design. The experts suspect that, checklist can help improve the quality of the documentation of the reference architecture, by helping to make it clear and complete. One of the other benefits cited were the flexibility, i.e. questions not considered suitable for the project scope can simply be dismissed. In addition, topics that may need more attention can be treated by new questions and questions can be modified to achieve a higher degree of suitability, depending on the evaluation team needs. The simplicity of the checklist was also highlighted, as it can be applied in little time due to the quantity of questions and its simplicity.

Regarding the downsides and disadvantages of using the checklist, the experts pointed that some of the targeted stakeholders may not be known in an early moment of

TABLE II
EXAMPLES OF QUESTION FROM THE CHECKLIST

Stage	Set	Respondent	Question
1	1	Architect	Does the reference architecture presents: overview information, release date, version, owner (e.g. organization), change history, short description, scope, domain terminology, open decisions and supporting material?
	2	Architect	Does the detail level favors the reference architecture understanding?
		Architect	Does each view correctly represent its viewpoint?
2	1	All Stakeholders	Do the selected viewpoints frame the concerns of all stakeholders (including domain-specific stakeholders)?
		All Stakeholders	For each viewpoint, are its models clear and well-defined? Do the models provide enough information for determining whether the concerns framed by the viewpoint have been satisfied?
		Architect	Can you show how you produced the list of RA stakeholders and their concerns?
		Domain Expert	Are the domain goals the system must satisfy clearly articulated and prioritized?
		Domain Expert	Is there traceability between the domain goals and the requirements the determine?
		Manager	Does the RA description allow an estimate of the effort for implementing it?
		Manager	Does the reference architecture description show what parts can be implemented using OTS or OSS components?
		Developer	Can you identify the allowed and prohibited dependencies between parts of the RA?
		Integrator	Do you understand the adaptation points of the RA?
		QA stakeholder	Is there a process to ensure conformance with quality attribute requirements?
		Tester	For each partition of the RA, can you determine what is needed (e.g., data, special hardware, other units) to test it?
		Analyst	If the RA is part of a life cycle or process that includes a procurement decision, does the RA contain the appropriate information to support the procurement process?
3	1	Architect	Is the current document complete in the sense that all information is documented? If not, are there placeholders for what has yet to be documented along with descriptions of what still needs to be worked out?
4	1	Domain Expert	How the power supply related decisions affect the reference architecture?
	2	Domain Expert	How embedded operating system considerations affect the reference architecture?

the reference architecture life cycle. The questions which target non existent stakeholders could, however, be simply discarded. Another downside was the difficulty to gather all the stakeholders for the application of the second stage. In this sense, FERA-ES, the tool currently under development should be able to overcome this limitation by making these gatherings asynchronous, which may extend the checklist application time, but without information loss.

In addition to the experts evaluation, the checklist was used to evaluate a reference architecture by Feitosa et al. [?]. It was applied to evaluate the SiMuS (from the portuguese, Sistemas Multirobóticos de Serviço - Multi-Robotic Service System) a reference architecture for multi-robotic services systems. The goal of this reference architecture is to support the architectural stage of the development of service robots that will perform tasks as a group [?]. The checklist was distributed among three evaluators, each of which received a specific set of questions based on their qualifications.

The evaluation was conducted by three evaluators in separate, that is, the evaluators did not collaborate with each

other to understand and answer the checklist. Despite that, the checklist application did not raise any doubts among any members of the evaluation team regarding the checklist. The evaluation team did not answer all the questions, however, some of them were not considered suitable for the level of abstraction of the reference architecture.

The results of this evaluations accused lack of information defects, i.e., the reference architecture did not express the variability points and how they affect the points that are fixed. The results also pointed that the SiMuS had no information about the relationship between views and stakeholders. On the other hand, the results showed that the abstraction level is adequate for the reference architecture purposes. Finally, the inspection showed that the SiMuS was ready for publishing but still needed reviews before adoption.

V. DISCUSSION

Results of our work point out that our checklist flexible and direct approach, the amount of questions is relatively

small and requires minimum effort to understand the questions and answer them. The inspection of reference architectures of embedded systems using the checklist can be conducted by parts, since only the second stage requires stakeholders to gather and discuss. The purpose of this discussion is to clarify doubts and provide an opportunity for all the stakeholders to understand the design choices and rationale.

Considering knowledge arisen from this work, it is possible to identify interesting and important research lines that can be investigated in future work. For example, the applicability of the FERA-ES for the checklist application. This tool could help the evaluation team to have a better control over the checklist application and good understanding of the results through the provided result management tool. Another possibility is the adaptation of the checklist to other reference architecture domains, aiming to prove its flexibility. This adaptation can be done with the replacement of the fourth stage, which holds questions specifically about the embedded systems domain. Thus, the change to other domain should not affect the appliance of the remaining stages and, therefore, should allow the application of the checklist to other domains.

Regarding limitation of this work, the checklist has only been tested in a research environment, and a validation in an industry case study could help demonstrate the checklist validity, applicability and pertinence. Also, the validation has not used a quantitative evaluation approach. Such evaluation would give a better idea of the improvement caused by the checklist use.

VI. CONCLUSIONS

The main contribution of this work was to present a way to evaluate reference architectures of embedded systems and demonstrate the applicability of this evaluation method. The checklist demonstrated to be an effective approach to catch defects in an early stage of the reference architecture life cycle, and help stakeholders understand better the reference architecture they are working with. As main result, we showed that the checklist is adequate for use, as considered by the experts, and also demonstrated in the checklist application on SiMuS. We also stated that the checklist is adaptable to other reference architecture domains, that is possible by replacing the last stage of the checklist.

We also presented future work possibilities: The checklist could be used to validate FERA-ES, which is currently under development, to assist its application and help disseminate it, it could be adapted to another domain to prove its flexibility. Another possibility is to conduct a case study on a reference architecture used in the industry.

Acknowledgments This work is supported by Brazilian funding agencies: FAPESP, CNPq and Capes.

REFERENCES

- [1] W. Wolf, *Computers as Components Principle of Embedded Computing System Design*. Morgan Kaufman, 2008.
- [2] J. C. Knight, "Safety critical systems: challenges and directions," in *ICSE*, 2002, pp. 547–550.
- [3] J.-L. Boulanger and V. Q. Dao, "Requirements engineering in a model-based methodology for embedded automotive software," in *RIVF*, 2008, pp. 263–268.
- [4] A. I. Wasserman, "Towards a discipline of software engineering," *IEEE Software*, vol. 13, no. 6, p. 23U31, 1996.
- [5] D. Garlan and D. Perry, "Introduction to the special issue on software architecture," *IEEE Transactions on Software Engineering*, vol. 21, pp. 269–274, 1995.
- [6] R. J. Allen, "A formal approach to software architecture," Ph.D. dissertation, Carnegie Mellon University, 1997.
- [7] D. C. Luckham, "Rhapsody: A language and toolset for simulation of distributed systems by partial orderings of events," in *Princeton University*, 1996.
- [8] Unified modeling language v. 2.3. (Accessed on 10/03/2013). [Online]. Available: <http://www.omg.org/spec/UML/2.3/>
- [9] J. Ivers, P. Clements, D. Garlan, R. Nord, B. Schmerl, and J. R. O. Silva, "Documenting component and connector views with uml 2.0," Software Engineering Institute (Carnegie Mellon University), Tech. Rep. CMU/SEI-2004-TR-008, 2004.
- [10] E. Y. Nakagawa, P. O. Antonino, and M. Becker, "Reference architecture and product line architecture: A subtle but critical difference," in *ECSCA*, 2011, pp. 207–211.
- [11] U. Eklund, O. Askerdal, J. Granholm, A. Alminger, and J. Axelsson, "Experience of introducing reference architectures in the development of automotive electronic systems," in *International workshop on Software engineering for automotive systems*, ser. SEAS '05. New York, NY, USA: ACM, 2005, pp. 1–6.
- [12] M. Guessi, "A contribution to the representation of reference architectures of embedded systems." Master's thesis, University of São Paulo, 2013.
- [13] ISO/IEC/(IEEE), "ISO/IEC 42010 (IEEE Std) 1471-2000 : Systems and Software engineering - Recommended practice for architectural description of software-intensive systems," 07 2007.
- [14] R. Kazman, G. Abowd, L. Bass, and P. Clements, "Scenario-based analysis of software architecture," *IEEE Software*, vol. 13, pp. 47–55, 1996.
- [15] E. Y. Nakagawa, "Uma contribuição ao projeto arquitetural de ambientes de engenharia de software." Ph.D. dissertation, University of São Paulo, 2006.
- [16] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford, *Documenting Software Architectures: Views and Beyond*, Addison-Wesley, Ed. Addison-Wesley, 2011.
- [17] N. Rozanski and E. Woods, *Software Systems Architecture - Working with stakeholders using viewpoints and perspectives*, Addison-Wesley Professional, 2011.
- [18] R. F. Barcelos and G. H. Travassos, "Arqcheck: Uma abordagem para inspeção de documentos arquiteturais baseada em checklist," in *V Simpósio Brasileiro de Qualidade de Software, SBQS 2006*, 2006, pp. 175 – 188.
- [19] E. Y. Nakagawa and J. C. Maldonado, "Reference architecture knowledge representation: An experience," in *SHARK*, 2008.
- [20] M. Guessi, E. Y. Nakagawa, F. Oquendo, and J. C. Maldonado, "Architectural description of embedded systems: a systematic review," in *ISARCS*, 2012, pp. 31–40.
- [21] D. Feitosa and E. Y. Nakagawa, "Simus - uma arquitetura de referência para sistemas multirrobóticos de serviço." Master's thesis, University of São Paulo, 2013.